name clears that up. And while many programming languages will suggest extracting methods, in Ruby you get the added bonus of making it possible to easily use the guard clause syntax in a separated method:

```
def gets_promotion_email?
  return false if last_active_date <= 1.year.ago
  return false if %w[IL GA MA].includes?(state)
  return false if opted_out?
  true
end

def send_promotion_email
  return false unless gets_promotion_email?
  send_the_email
end
```

There are a few things to note about this:

- You could do the gets_promotion_email as a single boolean expression as it was done in the first code snippet. We just find it easier to reason about each cause individually, and breaking them into a method makes that easier to do.
- It's also true that some of those clauses might be clearer with their own method. For example, the middle clause might use a method name to say why those states are blocked.
- The send_promotion_email method is now shorter and the meaning of the initial if clause is now clearer. The use of the end of line syntax, enabled by the fact that there's now just one method to call, is a signal that the first line is meant as a guard clause protecting the rest of the method.

Blocks have a similar mechanism. Many commonly used Ruby methods use blocks, and having the body of a block encapsulated in a method can enable you to use one of Ruby's shorter syntaxes, like this:

```
users.map { |user| user.convert_to_json }
users.map { _1.convert_to_json }
users.map(&:convert_to_json)
```

All three of these are equivalent, but they all depend on the body of the conversion being broken out into its own method.

Duck Typing

You may have noticed that in Ruby, you don't explicitly declare the types of variables or methods. Whether the particular value of a variable is correct for the messages being passed to it is evaluated at run time when the message is sent.

Folks tend to react to this in one of two ways. Some like this flexibility and feel comfortable writing code with dynamically typed variables and methods. Others get nervous when they think about all those objects floating around unconstrained. If you've come to Ruby from a language such as C#, Java, or TypeScript, where you're used to giving all your variables and methods an explicit type, you may feel that Ruby is just too permissive to use to write "real" applications.

It isn't.